Classes and Objects

What is a Class?

A class is a user-defined blueprint or template that describes the data members and methods of objects.

A class defines the properties (data) and behavior (functions) that its objects will have.

A class is a logical entity that encapsulates data members and member methods into a single unit.

Syntax:

```
class ClassName {
    // Data members
    int variable;

    // Member method
    void display() {
        System.out.println(variable);
    }
}
```

Example:

```
class Student {
    int rollNo;
    String name;

    void display() {
        System.out.println(rollNo);
        System.out.println(name);
    }
}
```

Here, `Student` is a class, `rollNo` and `name` are data members, and `display()` is a member method.

What is an Object?

An object is an instance of a class. It represents a real-world entity and has state and behavior.

- State: Data stored in the object's variables.
- Behavior: Actions performed through its methods.
- Identity: Distinguishes one object from another.

Syntax for creating an object:

```
ClassName objectName = new ClassName();
```

Example:

```
Student s1 = new Student();
```

Here, `s1` is a reference variable, and `new Student()` creates a `Student` object.

Complete Program: Class and Object

```
class Student {
    int rollNo;
    String name;

    void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }

    public static void main(String[] args) {
        Student s1 = new Student();

        s1.rollNo = 101;
        s1.name = "Rahul";

        s1.display();
    }
}
```

Output:

```
Roll No: 101
Name: Rahul
```

Explanation:

1. The `Student` class defines two variables and one method.
2. `new Student()` creates an object.
3. The dot (`.`) operator accesses its members.
4. `s1.display()` calls the object's method.

Difference Between Class and Object

Class	Object
A blueprint or template.	An instance of a class.
Defines variables and methods.	Uses the variables and methods defined by its class.
A class declaration does not itself create an instance.	An object is created at runtime in the usual object-creation process.
Example: `Student`	Example: `s1` refers to a `Student` object.
Defines a common structure.	Holds instance-specific state.

Advantages of Classes and Objects

- Code reusability.
- Better organization of programs.
- Encapsulation of data and behavior.
- Easier maintenance and debugging.
- Real-world entities can be represented in a program.

Methods in Java

A method is a block of code that performs a specific task. It executes when it is called, subject to the program's control flow.

Methods help divide a large program into smaller, manageable parts.

A method is a named block of statements defined inside a class to perform a particular operation.

Syntax of a Method

```
returnType methodName(parameterList) {
    // Statements
    return value;
}
```

Explanation of the components:

- `returnType`: Type of value returned by the method, such as `int`, `double`, or `void`.
- `methodName`: Name used to call the method.
- `parameterList`: Inputs accepted by the method.
- `return`: Sends a result back to the caller when the return type is not `void`.

Example:

```
int add(int a, int b) {
    return a + b;
}
```

This method accepts two integers and returns their sum.

Types of Methods

Methods can be classified in different ways.

A. Predefined Methods

These are methods already provided by Java's standard libraries.

Example:

```
System.out.println("Hello");
```

Here, `println()` is a predefined method.

Other examples include `Math.sqrt()`, `Math.max()`, and `String.length()`.

B. User-Defined Methods

These are methods created by the programmer.

Example:

```
class Demo {
    void display() {
        System.out.println("Welcome to Java");
    }
}
```

C. Methods Based on Arguments and Return Values

Type	Description	Example
No arguments, no return value	Accepts no input and returns nothing.	`void show()`
Arguments, no return value	Accepts input but returns nothing.	`void show(int x)`
No arguments, returns a value	Accepts no input but returns a result.	`int getNumber()`
Arguments and returns a value	Accepts input and returns a result.	`int add(int a, int b)`

Example of a Method with Arguments and Return Value

```
class Addition {
    int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        Addition obj = new Addition();

        int result = obj.add(10, 20);

        System.out.println("Sum = " + result);
    }
}
```

Output:

```
Sum = 30
```

Explanation:

1. `add()` accepts two integer parameters.
2. It calculates their sum.
3. `return a + b` returns the result.
4. The returned value is stored in `result`.

Method Calling

A method call transfers control to the specified method.

For an instance method:

```
objectName.methodName(arguments);
```

Example:

```
obj.add(10, 20);
```

For a static method:

```
ClassName.methodName(arguments);
```

Example:

```
Math.max(10, 20);
```

Method Overloading

Method overloading means defining multiple methods with the same name but different parameter lists in the same class.

The parameter lists can differ in the number, types, or order of parameters. Changing only the return type is not sufficient.

Example:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
```

```
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }

    public static void main(String[] args) {
        Calculator c = new Calculator();

        System.out.println(c.add(2, 3));
        System.out.println(c.add(2, 3, 4));
        System.out.println(c.add(2.5, 3.5));
    }
}
```

Output:

```
5
9
6.0
```

Advantages of methods:

- Reduce code duplication.
- Improve readability.
- Simplify testing and debugging.
- Support modular programming.
- Method overloading provides convenient names for related operations.

Constructors in Java

A constructor is a special member of a class used to initialize a newly created object.

A constructor has the same name as its class and has no return type, not even `void`.

Definition for exams: A constructor is a special class member that is invoked during object creation to initialize the object's state.

Characteristics of Constructors

1. Its name must be the same as the class name.
2. It has no return type.
3. It is invoked as part of object creation.
4. Constructors can be overloaded.
5. Constructors are not inherited.
6. A constructor cannot be declared `static`.
7. If no constructor is declared, Java may provide a default constructor automatically.
8. Constructors cannot be overridden like ordinary instance methods.

Types of Constructors

For introductory BCA examinations, the main types commonly discussed are:

- Default constructor.
- No-argument constructor.
- Parameterized constructor.

A. Default Constructor

A default constructor, strictly speaking, is the no-argument constructor supplied by the Java compiler when the programmer declares no constructor in the class.

Example:

```
class Student {
    int rollNo;
    String name;

    public static void main(String[] args) {
        Student s = new Student();

        System.out.println(s.rollNo);
        System.out.println(s.name);
    }
}
```

Output:

```
0
null
```

Explanation: The compiler supplies a default constructor because the class declares no constructor. The instance fields receive their default values: `0` for `int` and `null` for `String`.

Important: If you declare any constructor yourself, the compiler does not automatically provide the default constructor.

B. No-Argument Constructor

A no-argument constructor is a constructor explicitly written by the programmer that accepts no parameters.

Example:

```
class Student {
    Student() {
        System.out.println("Student object created");
    }

    public static void main(String[] args) {
        Student s = new Student();
    }
}
```

Output:

```
Student object created
```

C. Parameterized Constructor

A parameterized constructor accepts arguments to initialize an object's data members.

Example:

```
class Student {
    int rollNo;
    String name;

    Student(int r, String n) {
```

```
        rollNo = r;
        name = n;
    }

    void display() {
        System.out.println(rollNo + " " + name);
    }

    public static void main(String[] args) {
        Student s1 = new Student(101, "Amit");
        Student s2 = new Student(102, "Neha");

        s1.display();
        s2.display();
    }
}
```

Output:

```
101 Amit
102 Neha
```

Explanation: The constructor initializes each object using the values supplied during object creation.

Constructor Overloading

Constructor overloading means defining multiple constructors in the same class with different parameter lists.

Example:

```
class Box {
    int length;
    int breadth;

    Box() {
        length = 0;
        breadth = 0;
    }

    Box(int l, int b) {
        length = l;
        breadth = b;
    }

    void display() {
        System.out.println(length * breadth);
    }

    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = new Box(5, 4);

        b1.display();
        b2.display();
    }
}
```

Output:

```
0
20
```

Difference Between Constructor and Method

Constructor	Method
Initializes an object.	Performs a particular operation.
Name must match the class name.	Can have any valid method name.
Has no return type.	Has a return type, including `void`.
Invoked during object creation or constructor chaining.	Usually invoked through a method call.
Cannot be overridden.	Instance methods can be overridden when inheritance rules permit.
Cannot be `static`.	Can be `static` or non-static.

Advantages of Constructors

- Initialize objects automatically during creation.
- Reduce repeated initialization code.
- Support different ways of creating objects.
- Help establish valid initial object states.
- Improve code organization.

Static Members in Java

What is a Static Member?

A static member is a class-level member declared using the `static` keyword. It belongs to the class rather than to each individual object.

Static members include static variables, static methods, and static initialization blocks.

Definition for exams: Static members are class members that can be accessed using the class name without creating an object, subject to their access modifiers.

Static Variable

A static variable is also called a class variable. A single copy is shared by all instances of the class.

Example:

```
class Student {
    int rollNo;
    static String college = "MJPRU";

    Student(int r) {
        rollNo = r;
    }

    void display() {
        System.out.println(rollNo + " " + college);
    }

    public static void main(String[] args) {
        Student s1 = new Student(101);
        Student s2 = new Student(102);

        s1.display();
        s2.display();
    }
}
```

Output:

```
101 MJPRU
102 MJPRU
```

Explanation:

- `rollNo` is an instance variable. Each object has its own value.
- `college` is a static variable. Both objects share the same variable.
- Static variables are useful for data common to all objects of a class.

Static Method

A static method belongs to the class and can be called using the class name.

Example:

```
class Demo {
    static void display() {
        System.out.println("Static method called");
    }

    public static void main(String[] args) {
        Demo.display();
    }
}
```

Output:

```
Static method called
```

Rules of Static Methods

1. A static method can be called using the class name.
2. It can directly access static members of its class.
3. It cannot directly access instance variables or instance methods without an object reference.
4. It cannot use `this` or `super` directly.
5. A static method can be overloaded.
6. Static methods are hidden, rather than overridden polymorphically, when a subclass declares a matching static method.

Static Block

A static block is used to initialize static data. It runs when the class is initialized, before its first active use, subject to Java's class initialization rules.

Example:

```
class Demo {
    static int x;

    static {
        x = 100;
        System.out.println("Static block executed");
    }

    public static void main(String[] args) {
        System.out.println(x);
    }
}
```

Output:

```
Static block executed
100
```

Difference Between Static and Non-Static Members

Static Members	Non-Static Members
Belong to the class.	Instance members belong to individual objects.
Static variables have one shared class level copy per class.	Each object has its own instance variables.
Can be accessed using the class name when accessible.	Normally accessed through an object reference.
Static methods cannot directly use `this`.	Instance methods can use `this`.
Can be accessed without creating an object, where appropriate.	Instance members require an object instance for access.

Advantages of Static Members

- Share common data among objects.
- Avoid storing the same class-level value separately in every instance.
- Provide utility methods, such as `Math.max()`.
- Support program initialization through static blocks.
- Allow convenient access to class-level functionality.

Example: Counting Objects Using a Static Variable

This is an important program for practice.

```
class Student {
    static int count = 0;

    Student() {
        count++;
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student();
        Student s3 = new Student();

        System.out.println("Total objects = " + Student.count);
    }
}
```

Output:

```
Total objects = 3
```

Explanation: The constructor increments the shared static variable whenever a new object is created. After three object creations, `count` becomes `3`.

Nesting of Methods in Java

In introductory Java theory, nesting of methods generally refers to one method calling another method to perform a task.

Java does not allow an ordinary named method to be declared inside another method. Instead, one method can invoke another method within the same class.

Definition for exams: Nesting of methods is the process in which one method calls another method to perform a specific operation.

Example of Nesting of Methods

```
class Nesting {
    void methodOne() {
        System.out.println("Method One");
        methodTwo();
    }

    void methodTwo() {
        System.out.println("Method Two");
        methodThree();
    }

    void methodThree() {
        System.out.println("Method Three");
    }

    public static void main(String[] args) {
        Nesting obj = new Nesting();
        obj.methodOne();
    }
}
```

Output:

```
Method One
Method Two
Method Three
```

Explanation

1. The `main()` method calls `methodOne()`.
2. `methodOne()` prints a message and calls `methodTwo()`.
3. `methodTwo()` prints a message and calls `methodThree()`.
4. `methodThree()` prints the final message.
5. After `methodThree()` completes, control returns through the calling methods.

Flow of Nested Method Calls

main()

methodOne()

methodTwo()

methodThree()

Control returns in reverse order when each called method finishes.

Advantages of Nesting of Methods

- Avoids repetition of code.
- Improves modularity.
- Makes programs easier to understand.
- Allows complex tasks to be divided into smaller tasks.
- Simplifies testing and debugging.

Important Note: Nesting vs. Recursion

These concepts are related but different.

Nesting of Methods	Recursion
One method calls another method.	A method calls itself directly or indirectly.
Different methods may perform different tasks.	Repeated calls solve a problem through smaller or simpler cases.
Does not necessarily require a termination condition for recursion.	Requires an appropriate base case to terminate recursive calls.
Example: `methodOne()` calls `methodTwo()`.	Example: `factorial()` calls `factorial()` again.

Quick Revision Sheet

Topic	Key point to remember
Class	Blueprint of objects.
Object	Instance of a class.
Method	Performs a particular operation.
Method overloading	Same method name, different parameter lists.
Constructor	Initializes a newly created object.
Default constructor	Compiler-supplied when no constructor is declared.
No-argument constructor	Programmer-defined constructor with no parameters.
Parameterized constructor	Constructor that accepts parameters.
Static variable	Shared class-level variable.
Static method	Class-level method.
Static block	Used for static initialization.
Nesting of methods	One method calls another method.

Inheritance and Extending a Class

Inheritance is an OOP feature in which one class acquires accessible fields and methods from another class. It supports code reusability and establishes a relationship between classes.

- Parent class / Superclass / Base class: The class whose members are inherited.
- Child class / Subclass / Derived class: The class that inherits from another class.

Inheritance is the mechanism by which one class acquires the properties and behaviors of another class using the `extends` keyword in Java.

Syntax of Inheritance

```
class Parent {
    void display() {
        System.out.println("Parent class");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child class");
    }
}
```

Here, `Child` extends `Parent` and inherits its accessible members.

1.3 Example of Inheritance

```
class Animal {
    void eat() {
```

```
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

class Test {
    public static void main(String[] args) {
        Dog d = new Dog();

        d.eat();
        d.bark();
    }
}
```

Output:

```
Animal is eating
Dog is barking
```

Explanation:

1. `Animal` is the superclass.
2. `Dog` is the subclass.
3. `Dog` extends `Animal` using the `extends` keyword.
4. The `Dog` object can call both `eat()` and `bark()`.

Types of Inheritance in Java

1. Single Inheritance

One child class inherits from one parent class.

Example: `class Dog extends Animal`

2. Multilevel Inheritance

A class inherits from a class that already inherits from another class.

Example: `A → B → C`

3. Hierarchical Inheritance

Multiple child classes inherit from the same parent class.

Example: `Dog` and `Cat` extend `Animal`.

4. Multiple Inheritance

One class inherits features from multiple parents. Java does not support multiple inheritance of classes, but a class can implement multiple interfaces.

Advantages of Inheritance

- Promotes code reusability.

- Reduces code duplication.
- Supports method overriding.
- Helps implement runtime polymorphism.
- Makes programs easier to extend and maintain.

What Is the `extends` Keyword?

The `extends` keyword is used to create a subclass from a superclass.

Example:

```
class A {
    int x = 10;
}

class B extends A {
    int y = 20;
}
```

An object of `B` can access both `x` and `y` when the members are accessible.

```
B obj = new B();
System.out.println(obj.x);
System.out.println(obj.y);
```

Output:

```
10
20
```

Constructors are not inherited, and private members cannot be accessed directly by a subclass. Java also does not allow a class to extend more than one class.

Method Overriding

Method overriding occurs when a subclass provides its own implementation of an inherited instance method, using the same method signature and a compatible return type.

It is used to achieve runtime polymorphism in Java.

Rules of Method Overriding

1. Inheritance must exist between the superclass and subclass.
2. The method must have the same name and parameter types.
3. The return type must be the same or compatible according to Java's covariant return-type rules.
4. The overriding method cannot have more restrictive access than the overridden method.
5. `final` methods cannot be overridden.
6. Static methods are hidden, not overridden.
7. Private methods cannot be overridden because they are not inherited as accessible methods.
8. The `@Override` annotation is recommended because it helps the compiler detect mistakes.

Example of Method Overriding

```
class Animal {
```

```
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Test {
    public static void main(String[] args) {
        Animal a = new Animal();
        Dog d = new Dog();
        Animal ref = new Dog();

        a.sound();
        d.sound();
        ref.sound();
    }
}
```

Output:

```
Animal makes a sound
Dog barks
Dog barks
```

Explanation:

- `Animal` defines the original `sound()` method.
- `Dog` overrides `sound()` with a new implementation.
- `Animal ref = new Dog();` stores a reference to a `Dog` object in an `Animal` reference.
- At runtime, the overridden method of `Dog` executes.

This is an example of runtime polymorphism.

Difference Between Method Overloading and Overriding

Method Overloading	Method Overriding
Same method name with different parameter lists.	Same inherited instance-method signature.
Can occur within the same class.	Requires inheritance between classes.
Compile-time polymorphism.	Runtime polymorphism for instance methods.
Return type alone cannot distinguish overloaded methods.	Return type must be compatible with the overridden method.
Example: `add(int, int)` and `add(int, int, int)`.	A subclass redefines `sound()`.

Advantages of Method Overriding

- Provides specialized behavior in subclasses.
- Supports runtime polymorphism.
- Improves flexibility and extensibility.
- Allows the same method call to behave differently for different object types.

Exam tip: Always explain the parent class, child class, overriding rules, a complete program, and its output when answering a long question.

Final Variables and Final Methods

In Java, the keyword `final` is used to restrict modification or extension. Its meaning depends on where it is applied.

Final Variable

A final variable is a variable that can be assigned only once. After it has been initialized, it cannot be reassigned.

Syntax:

```
final dataType variableName = value;
```

Example:

```
class Demo {
    public static void main(String[] args) {
        final int MAX = 100;

        System.out.println(MAX);

        // MAX = 200; // Compilation error
    }
}
```

Output:

```
100
```

The statement `MAX = 200` is illegal because `MAX` is final.

Important Points About Final Variables

- A final local variable must be assigned before it is used.
- A final instance variable can be initialized at declaration or in a constructor or instance initializer.
- A blank final variable is declared without an initial value and assigned exactly once through an allowed initialization path.
- A `static final` variable is commonly used to declare constants.
- If a final variable stores an object reference, the reference cannot be reassigned, but the object's mutable state may still be changed.

Example:

```
final int AGE = 20;
```

The value of `AGE` cannot be changed after initialization.

Static Final Variable

A `static final` variable is commonly used for a class-level constant.

Example:

```
class Circle {
    static final double PI = 3.14159;

    public static void main(String[] args) {
```

```
        double radius = 5;
        double area = PI * radius * radius;

        System.out.println(area);
    }
}
```

Output:

```
78.53975
```

Here, `PI` is a constant shared at the class level.

Final Method

A final method is a method that cannot be overridden in a subclass.

Syntax:

```
final void display() {
    // Statements
}
```

Example:

```
class Parent {
    final void show() {
        System.out.println("Final method");
    }
}

class Child extends Parent {
    // Cannot override show()
}

class Test {
    public static void main(String[] args) {
        Child c = new Child();
        c.show();
    }
}
```

Output:

```
Final method
```

If `Child` declares another instance method with the same signature as `show()`, the program will produce a compilation error because `show()` is final.

Rules of Final Methods

1. A final method cannot be overridden.
2. A final method can be inherited and called if it is accessible.
3. A final method can be overloaded.
4. A final method may be either static or non-static.
5. A final method cannot be abstract because a final method cannot require a subclass implementation.

Difference Between Final Variable and Final Method

Final Variable	Final Method

Prevents reassignment of the variable.	Prevents overriding of the method.
Declared using `final` before the variable type.	Declared using `final` in the method declaration.
Commonly used for constants.	Used when subclass implementations must not replace the method.
Example: `final int x = 10;`	Example: `final void show() { }`

Final Classes

A final class is a class that cannot be extended by another class.

The `final` keyword prevents inheritance from that class.

Syntax:

```
final class ClassName {
    // Members
}
```

Example of a Final Class

```
final class Vehicle {
    void display() {
        System.out.println("Vehicle class");
    }
}

class Test {
    public static void main(String[] args) {
        Vehicle v = new Vehicle();
        v.display();
    }
}
```

Output:

```
Vehicle class
```

The program works because `Test` creates an object of `Vehicle`; it does not attempt to extend it.

If we write:

```
class Car extends Vehicle {
}
```

the compiler reports an error because `Vehicle` is final.

Examples of Final Classes in Java

- `String`
- Wrapper classes such as `Integer` and `Double`

These classes cannot be extended by user-defined subclasses.

Advantages of Final Classes

- Prevent unwanted subclassing.

- Help preserve intended class behavior.
- Can support the design of immutable classes.
- Improve control over class hierarchies.

Important: Declaring a class `final` does not automatically make all its fields final or its objects immutable.

Difference Between Final Variable, Method, and Class

Keyword applied to	Meaning
Final variable	Cannot be reassigned after initialization.
Final method	Cannot be overridden.
Final class	Cannot be extended.

This is a common short-answer or comparison question for exam preparation.

`finalize()` Method in Java

The `finalize()` method was historically used to perform cleanup before an object was reclaimed by the garbage collector.

Exam definition: The `finalize()` method was a method of the `Object` class that the garbage collector could invoke before reclaiming an object, giving the object an opportunity to perform cleanup.

Important modern Java note: The `finalize()` method has been deprecated since Java 9 and deprecated for removal. Its execution is not guaranteed, so it should not be relied upon for resource cleanup.

Syntax

```
protected void finalize() throws Throwable {
    // Cleanup code
}
```

Example for Understanding

The following program illustrates the historical use of `finalize()`. Its output is not guaranteed in modern Java.

```
class Demo {
    @Override
    protected void finalize() throws Throwable {
        System.out.println("Finalize method called");
    }

    public static void main(String[] args) {
        Demo obj = new Demo();

        obj = null;
        System.gc();

        System.out.println("End of main");
    }
}
```

Possible output:

```
End of main
```

```
Finalize method called
```

The order may differ, and the `finalize()` message may never appear. Calling `System.gc()` only requests garbage collection; it does not guarantee that collection or finalization will occur.

Characteristics of finalize()

1. It was defined in the `Object` class.
2. It was historically associated with garbage collection.
3. It could be overridden by a subclass.
4. It was not guaranteed to execute.
5. Its timing was unpredictable.
6. It is deprecated and should not be used in new Java programs.

Limitations of finalize()

- Execution is unpredictable.
- Cleanup may never occur.
- It can introduce performance and security problems.
- It can delay garbage collection.
- It is unsuitable for reliable resource management.

Modern alternative: Use `try-with-resources` and `AutoCloseable` for resources such as files and streams. Use explicit cleanup methods where appropriate.

Difference Between final, finally, and finalize()

This is an important Java theory question.

`final`	`finally`	`finalize()`
A keyword.	A block used with exception handling.	A deprecated method of `Object`.
Restricts reassignment, overriding, or inheritance depending on its use.	Usually executes after `try`/`catch` processing.	Historically associated with garbage collection.
Used with variables, methods, and classes.	Used with exception-handling statements.	Could be overridden by a subclass.
Example: `final int x = 10;`	Example: `finally { cleanup(); }`	Example: `protected void finalize() { }`

Remember that a `finally` block is not absolutely guaranteed to execute in every possible circumstance, such as abrupt JVM termination.

Abstract Methods and Abstract Classes

What Is an Abstract Method?

An abstract method is a method declared without an implementation in the class where it is declared.

It is declared using the `abstract` keyword and ends with a semicolon.

An abstract method is a method that specifies its signature but does not provide a method body, requiring a concrete subclass to implement it unless the subclass is also abstract.

Syntax:

```
abstract returnType methodName();
```

Example:

```
abstract void display();
```

The method declares what operation must be provided, but not how it is performed.

What Is an Abstract Class?

An abstract class is a class declared using the `abstract` keyword that cannot be instantiated directly.

It may contain abstract methods, concrete methods, constructors, instance variables, and static members.

An abstract class is a class that cannot be instantiated directly and can provide a common base for subclasses through abstract and concrete methods.

Syntax:

```
abstract class ClassName {
    abstract void display();

    void show() {
        System.out.println("Concrete method");
    }
}
```

Example of an Abstract Class and Abstract Method

```
abstract class Shape {
    abstract void draw();

    void message() {
        System.out.println("Drawing a shape");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a circle");
    }
}

class Test {
    public static void main(String[] args) {
        Circle c = new Circle();

        c.message();
        c.draw();
    }
}
```

Output:

```
Drawing a shape
Drawing a circle
```

Explanation

1. `Shape` is an abstract class.
2. `draw()` is an abstract method without a body.
3. `message()` is a concrete method with a body.
4. `Circle` extends `Shape`.
5. `Circle` implements the abstract method `draw()`.
6. An object of `Circle` can be created, but an object of `Shape` cannot be created directly.

For example, `Shape s = new Shape();` is illegal, while `Shape s = new Circle();` is valid.

Rules of Abstract Methods

1. An abstract method is declared with the `abstract` keyword.
2. It has no method body and ends with a semicolon.
3. A concrete subclass must implement inherited abstract methods unless it is itself declared abstract.
4. An abstract method cannot be `final`.
5. An abstract method cannot be `static`.
6. An abstract method cannot be `private`, because it must be available for implementation by subclasses.
7. Abstract methods can have different valid access levels, including `public` and `protected`.

Rules of Abstract Classes

1. An abstract class cannot be instantiated directly.
2. It may contain both abstract and concrete methods.
3. It can have constructors.
4. It can have instance variables and static members.
5. An abstract class can extend another class and implement interfaces.
6. An abstract class can have no abstract methods at all.
7. A subclass must implement all inherited abstract methods to be concrete.

Real-Life Example of an Abstract Class

Consider a payment system. Different payment methods perform payments differently, but all share a common operation called `pay()`.

```
abstract class Payment {
    abstract void pay(double amount);
}

class UpiPayment extends Payment {
    @Override
    void pay(double amount) {
        System.out.println("Paid using UPI: " + amount);
    }
}

class CardPayment extends Payment {
    @Override
    void pay(double amount) {
        System.out.println("Paid using card: " + amount);
    }
}

class Test {
    public static void main(String[] args) {
        Payment p1 = new UpiPayment();
        Payment p2 = new CardPayment();
```

```
        p1.pay(500);
        p2.pay(1000);
    }
}
```

Output:

```
Paid using UPI: 500.0
Paid using card: 1000.0
```

This demonstrates abstraction and runtime polymorphism: the common abstract class defines the required operation, while each subclass supplies its implementation.

Difference Between Abstract Class and Concrete Class

Abstract Class	Concrete Class
Cannot be instantiated directly.	Can be instantiated if it is otherwise accessible and valid.
May contain abstract methods.	Cannot contain unimplemented abstract methods.
Can contain abstract and concrete methods.	Contains implemented methods.
Often serves as a base class.	Often provides a complete implementation.
Declared with `abstract`.	Does not need the `abstract` keyword.

Difference Between Abstract Class and Interface

Abstract Class	Interface
Declared using `abstract class`.	Declared using `interface`.
Can have instance variables.	Fields are implicitly `public static final`.
Can have constructors.	Cannot have constructors.
Can contain abstract and concrete methods.	Can contain abstract methods, default methods, static methods, and private helper methods.
A class can extend only one class.	A class can implement multiple interfaces.

Quick revision table

Topic	Key point
Inheritance	Acquiring accessible members from a parent class.
`extends`	Used for class inheritance.
Overriding	Subclass provides a compatible implementation of an inherited instance method.
Final variable	Cannot be reassigned after initialization.
Final method	Cannot be overridden.
Final class	Cannot be extended.
`finalize()`	Deprecated; unreliable for cleanup.
Abstract method	Declared without a method body.
Abstract class	Cannot be instantiated directly; can provide a common base for subclasses.